

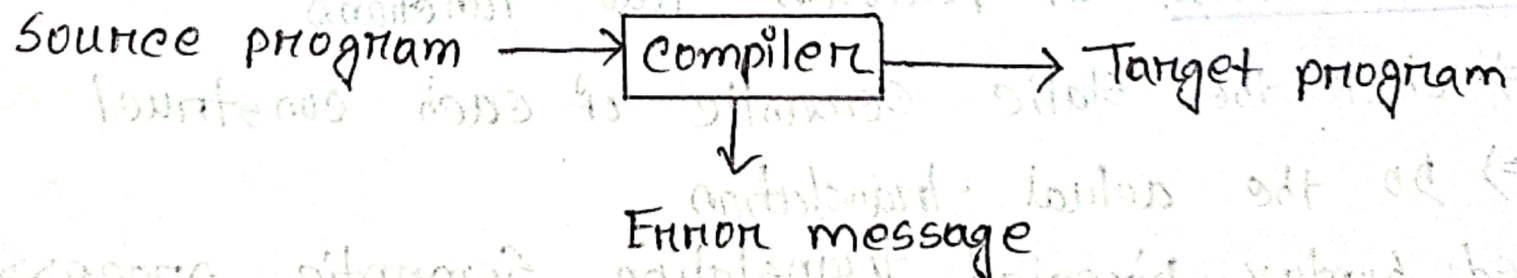
Compiler: A compiler is a program that reads a program written in one language (the source language) and translates it into an equivalent program in another language (the target language).

⇒ A compiler is a translator that converts the HLL (high level language) into the machine language.

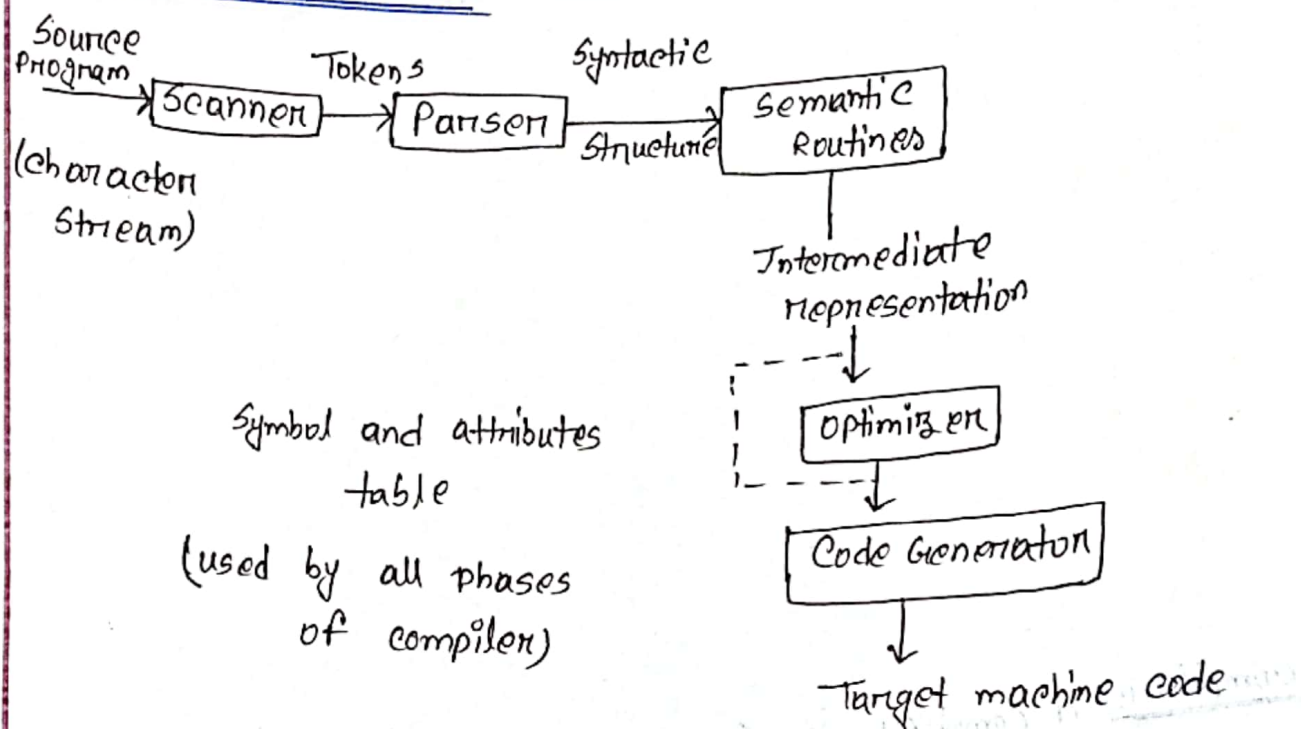
⇒ Compiler is used to show errors to the programmer.

⇒ The main purpose of compiler is to change the code written in one language without changing the meaning of the program.

Schematic diagram of compiler:



Structure of compiler:



Scanner: The scanner begins the analysis of the source program by reading the input, character by character, and grouping character into individual words and symbols which are called Token.

used RE (regular expression), NFA, DFA (Deterministic finite Automata), LEX

Parser: The parser reads tokens and groups them into units as specified by the production of the CFG being used: CFG, BNF (Backus Normal Form), GAA (Grammar analysis algorithms), LL, SLR etc.

Semantic Routines: They perform two functions.

⇒ Check the static semantic of each construct

⇒ Do the actual translation

used: Syntax Directed translation, Semantic processing techniques, IR (Intermediate representation)

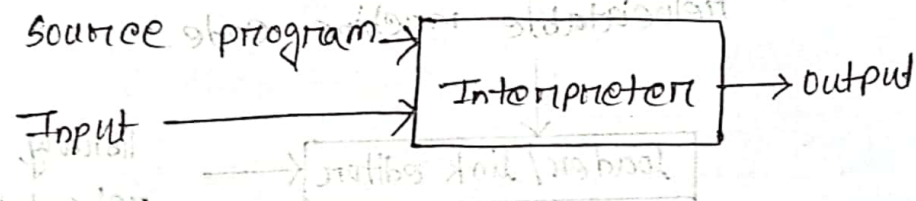
Optimizer: The IR code generated by the semantic routine is analyzed and transformed into functionally equivalent but improved IR code.

→ This phase can be complex and slow.

used: Registers and temporary management

Code Generator: Interpretive code generation, generating code for tree.

Interpreter: An interpreter appears to directly execute the operation specified in the source program on inputs supplied by the user.



Benefits:

- The machine language ~~the~~ target program produced by a compiler is usually much faster.

- Interpreter can give better error diagnostics than a compiler, because it executes the source program statement by statement.

Language processing system:

Skeletal source program

↓
Preprocessor

↓
Source program

↓
Compiler

↓
target assembly program

↓
assembler

↓
relocatable machine code

↓
loader/link editor

library,
relocatable object
files

↓
Absolute machine code

Fig: A language processing system

we write the program in high level language, those programs are then fed into a series of tools and os components to get desired code that can be used by the machine which is known as Language processing system.

Preprocessor: A preprocessor, generally considered as a part of compiler, is a tool that produces input for compilers.

⇒ It deals with micro-processing, augmentation, file inclusion, language extension etc.

Assembler: ⇒ An assembler translates assembly language programs into machine code.

⇒ The output of an assembler is called an object file.

⇒ It contains a combination of machine instruction.

An assembler is a type of computer program that takes in basic instructions and converts them into a pattern of bits that the computer's processor can use to perform basic operations.

Linker: Linker is a computer program that links and merges various object files together to make an executable file.

⇒ The major task is to search and locate referenced module in a program.

⇒ To determine all memory location.

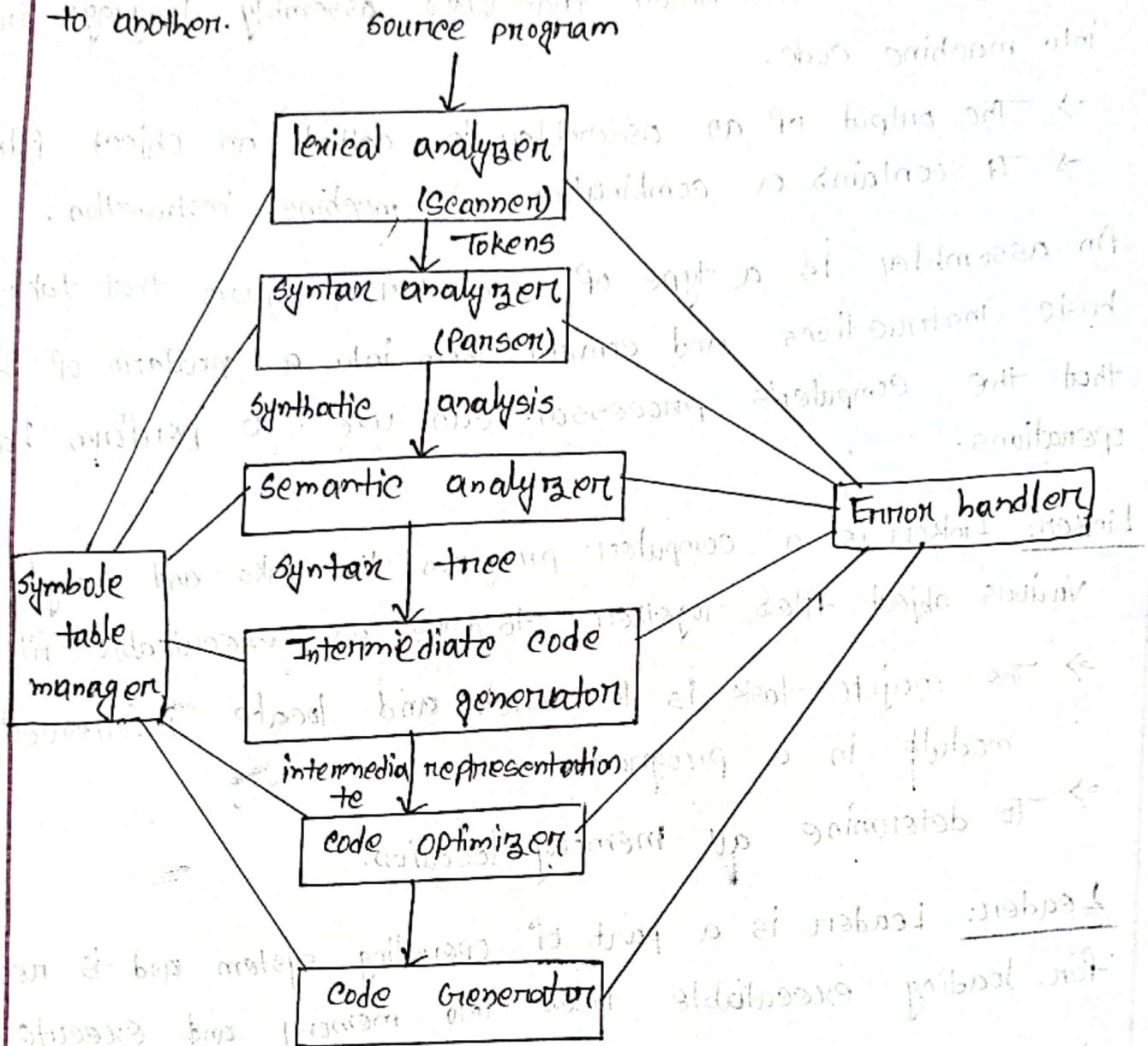
Loader: Loader is a part of operating system and is responsible for loading executable files into memory and execute them.

⇒ It calculates the size of a program (instruction and data)

⇒ It encodes memory space.

⇒ It initializes various registers.

Phases of compiler: A compiler operates in phases each of which transforms the source program from one representation to another.



Symbol table management: A symbol table is a data structure containing a record for each identifier.

⇒ It allows the compiler to efficiently store and retrieve information about identifier.

⇒ When an identifier in the source program is detected by lexical analyzer, the identifier is entered into the symbol table.

Error handling: The task of the error handling process are to detect each error, report it to the user and then make some recovery strategy and implement them to handle the error.

Analysis of the source program: In this analysis, compiling, analysis consists of three phases:

① Linear analysis: In this analysis, the stream of characters making up the source program is read from left to right and grouped into tokens the sequences of characters having collective meaning.

② Hierarchical analysis: In this analysis, characters or tokens are grouped hierarchically into nested collection with collective meaning.

③ Semantic analysis: In this analysis, certain checks are performed to ensure that the components of a program fit together meaningfully.

Content of compiler: A source program may be divided into modules stored in separate files. The task of collecting the source program is sometimes entrusted to a distinct program called a preprocessor. The preprocessor may also expand shorthands, called macros. The target program created by the compiler may require further processing before it can be run.

⇒ The compiler creates assembly code that is translated by an assembler into machine code and then linked together with some library to the code and runs on the machine.

Difference of analysis: Diffn between Lexical analyser and syntax analyser:

Lexical analyser	Syntax analyser
① Scan input program	① Perform syntax analysis
② Identify tokens	② Create an abstract representation of the code
③ Invent token into symbol token	③ update to symbol table entries.
④ It generates lexical errors.	④ It generates a parse tree of the source code.

Lexical analysis: The 1st phases of compiler is called lexical analysis or scanning.

The lexical analyser begins the analysis of the source program by reading the input character by character and grouping character into individual words and symbol called tokens.

$$\text{Position} = \text{initial} + \text{rate} * 60$$

would be grouped into following tokens:

1. The identifier position

2. The assignment symbol : =

3. The identifier initial

4. The plus sign

5. The identifier rate

6. The multiplication sign

7. The number 60

Function of Lexical analysis:

1. Convert the source code into stream of tokens
2. Removing white spaces
3. Removing comments
4. Recognizing keywords and identifiers
5. Recognizing constants
6. Show error when lexem doesn't match any pattern.

Error recovery in lexical analysis:

1. Removes one character from the remaining input.
2. In panic mode, the successive characters are always ignored until we reach a well formed token.
3. By inserting and missing character into the remaining input
4. Replace a character with another character.
5. Transpose two serial character.

Lexeme: A lexeme is a sequence of characters that are included in the source program. According to the matching pattern of a token.

Token: Token is a compiler design are the sequence of characters which represents a unit of information in the source program.

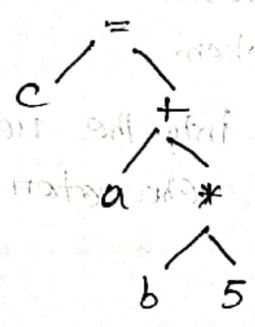
Pattern: pattern is a description which is used by the token.

Syntax Analysis: Syntax analysis is the 2nd phase of compiler which is also called as parsing. Parser converts tokens produced by lexical analyzer into a tree like

representation called parse tree.

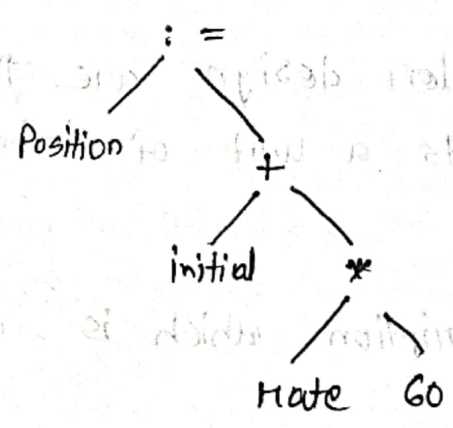
- ⇒ The parser uses the 1st components of the tokens produced by the lexical analyzer to create a tree.
- ⇒ A typical representation is a syntax tree^m in which each interior node represents an operation and the children of the node represent the arguments of the operation.

$$c = a + b * 5$$



Semantic analysis

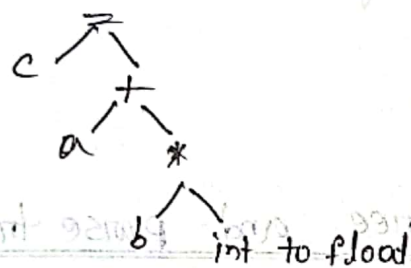
Parse tree for position: = initial + rate * 60



Semantic analysis: Semantic analysis is the 3rd phase of compiler.

- ⇒ The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program.
- ⇒ It gathers type information and saves it in either syntax tree or in the symbol table.
- ⇒ An important part of semantic analysis is type checking.
- ⇒ It checks the semantic consistency.

$C = a + b * 5$



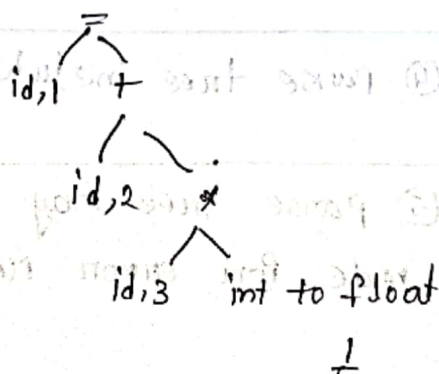
Intermediate code generation: Intermediate code generations produces intermediate representations for the source program which are of the following terms,

Postfix notation

Three address code

Syntax tree

Code



$t_1 = \text{int to float}(5)$

$t_2 = id_3 * t_1$

$t_3 = id_2 + t_2$

$id_1 = t_3$

Code optimization: The machine independent code-optimization phase attempts to improve the intermediate code so that better target code will result.

Target code generation: The code generator takes as input an intermediate representation of the source program and maps it into the target language.

→ A crucial aspect of code generation is the judicious assignment of registers to hold variables

Difference between syntax tree and parse tree:

Syntax tree	Parse tree
① A syntax tree is created by the compiler based on the parse tree after the parser finished processing the source code.	① A parse tree is created by Parser which is a component of a compiler that processes the source code.
② Syntax trees are simpler & more succinct.	② Parse trees are typically more detailed.
③ Syntax trees traverse the final representation used by the compiler to generate machine code.	③ Parse trees are used on intermediate representation.
④ Syntax trees do not include information.	④ Parse trees include information.
⑤ Syntax trees do not include nodes.	⑤ Parse trees may include node for error recovery.

Example: Find token numbers:

```

1 /* Find sum of x and y */
2 int sum(x, y) {
3     return x+y;
4 }

```

tokens: 14

```

5 printf("sum = %d\n", sum);

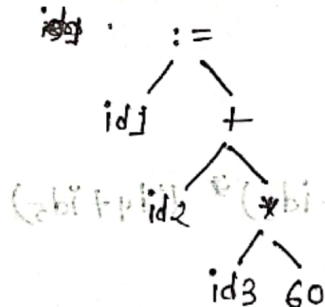
```

tokens: 7

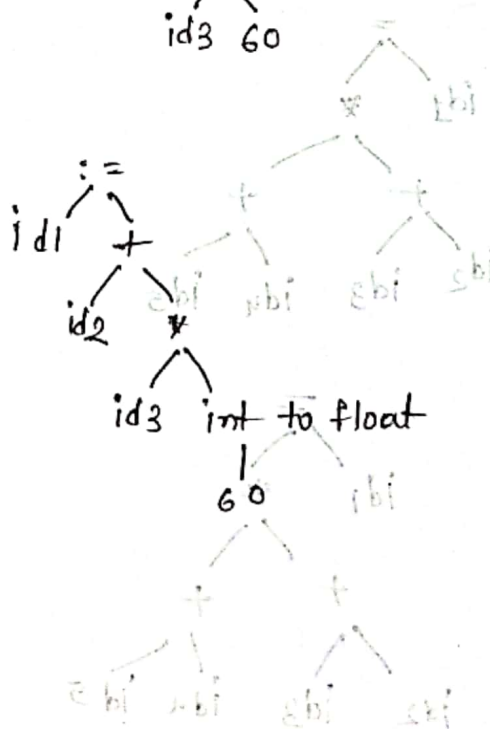
Example: Position := initial + rate * 60

Lexical analysis: id1: id2 + id3 * 60

Syntax analysis:



Semantic analysis:



abstract syntax tree with attributes

Intermediate code generation:

temp₁ := into read (60)

temp₂ := id₃ * temp₁

temp₃ := id₂ + temp₂

id₁ := temp₃

Code optimization: temp₁ := id₃ * 60.0

id₁ := id₂ + temp₁

Target code generation:

MOV id₃, R₂

MUL F R₂, #60.0

MOV F id₂, R₁

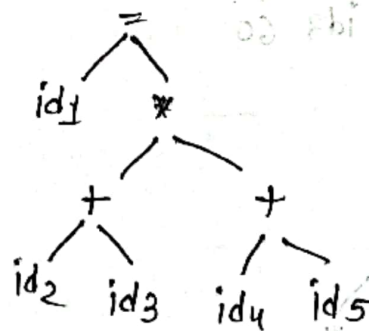
ADD F R₂, R₁

MOV F R₁, id₁

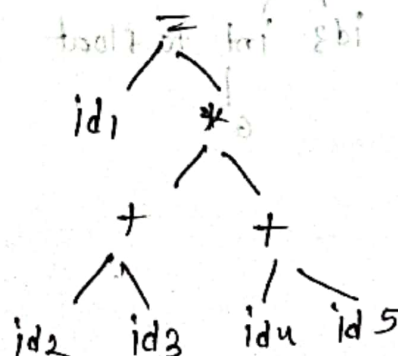
Q) $x = (a+b) * (c+d)$

Lexical analysis: id₁ = (id₂ + id₃) * (id₄ + id₅)

Syntax analysis:



Semantic analysis:



Intermediate code generator:

$$t_1 = id_2 + id_3$$

$$t_2 = id_4 + id_5$$

$$t_3 = t_1 * t_2$$

$$id_1 = t_3$$

Code optimization:

$$t_1 = id_2 + id_3$$

$$t_2 = id_4 + id_5$$

$$id_1 = t_1 * t_2$$

Code generator:

LDF R_1, id_2

ADD $F R_1, R_1, id_3$

LDF R_2, id_4

ADD $F R_2, R_2, id_5$

MUL $F R_1, R_1, R_2$

STF id_1, R_1

Compiler construction tools: we use software tools such as debuggers, version managers, profilers and so on.

Some tools have been created for automatic design of specific compiler components. These tools use specialised language for specifying and implementing the component:

- ① Parser generator: These produce syntax analyzers normally from input that is based on CFG. This phase is now considered one of the easiest to implement. Many parser generators utilize powerful parsing algorithms that are too complex.
- ② Scanner generator: These generate lexical analyzers which based on regular expression. ~~It~~ It is a finite automaton.
- ③ Syntax-directed translation engines: These produce collection of routines that walk the parse tree, generating intermediate code. The basic idea ~~of~~ is translations are associated with each node of parse tree.
- ④ Automatic code generator: Each operation of the intermediate language into the machine language for the target language. The basic technique is template matching. The intermediate code replace by templates.
- ⑤ Data flow engines: Much of the information needed to perform good code optimization involve data-flow analysis.

Abstract syntax tree: An AST is usually the result of the syntax analysis phase of a compiler. It often serves as an intermediate representation of the program. It has a strong impact on the final output of the compiler.

Concrete syntax tree: A concrete syntax tree represents the source text exactly in parsed form. It conforms to the CFG defining the source language.

Difference between abstract and concrete syntax tree:

Abstract syntax tree	Concrete syntax tree
① AST (Abstract syntax tree)	① CST (concrete syntax tree)
② Representation of grammars in a tree. the source code.	② Representation of the source code . grammars in a tree.
③ AST are more concrete abstract.	③ CST are more concrete.
④ Abstract tree is a variation of parse tree.	④ A parse tree is often also called a concrete syntax tree.

position := initial + rate * 60

SYMBOL TABLE		
1	position	...
2	initial	...
3	rate	...
4		

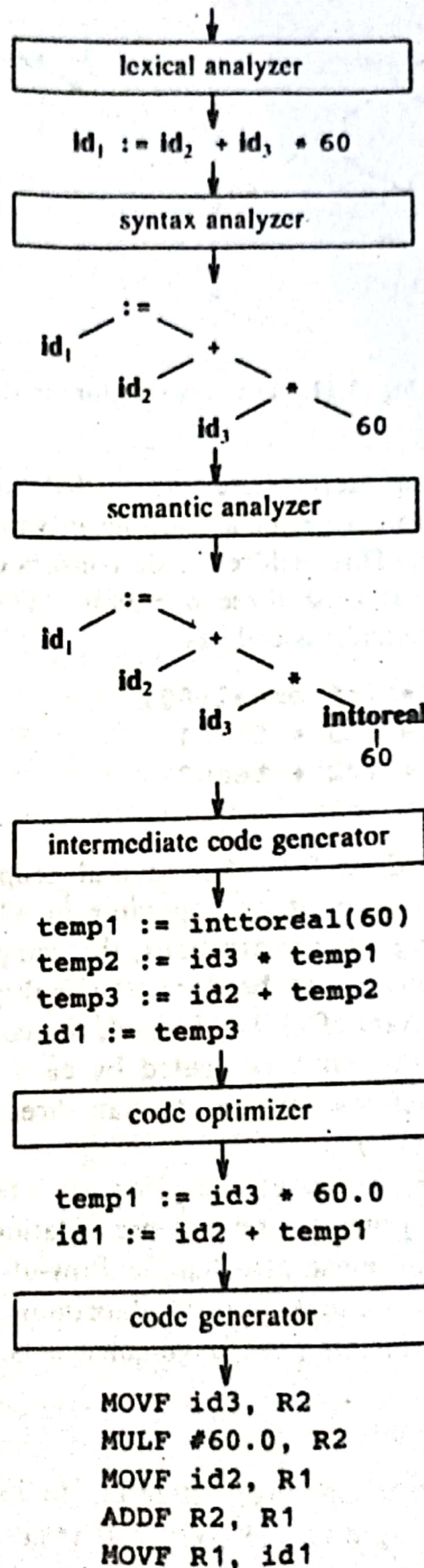


Fig. 1.10. Translation of a statement.